

Age and Emotion Detection through Voice

From a Self-Built Classical Pipeline to a Deep-Learning Upgrade

Machine Learning Project Report

librosa · scikit-learn · wav2vec2 (PyTorch) · Streamlit

What's in here

1. The task
2. How I broke it down
3. Phase 1 — the classical pipeline (app.py)
 - 3.1 Gender model
 - 3.2 Age model
 - 3.3 Emotion model
 - 3.4 How well Phase 1 actually worked
4. Why Phase 1 wasn't good enough
5. Phase 2 — the deep-learning version (app2.py)
 - 5.1 What wav2vec2 is and why I moved to it
 - 5.2 Plugging it in
 - 5.3 Benchmarking it against my own model
6. The two apps, side by side
7. The interface
8. Things that went wrong along the way
9. Results at a glance
10. Where I landed, and what's next

1. The task

The brief was to detect a person's age from a voice note, with a few rules wrapped around it. The system only handles male voices — if a female voice comes in, it has to reject it and say "Upload male voice." If the speaker is over 60, it marks them as a senior citizen and also works out their emotion. If they're under 60, it just reports the age and stops there.

The brief also said this was mainly a test of logic-building and problem-solving, that the model had to be my own, and that it needed a GUI. It said evaluation would be based on how the whole thing behaves and whether the interface works, not just on a raw accuracy number. I kept that in mind the whole way through, because it changes what "done" means — a clean, well-reasoned pipeline matters more here than chasing one big percentage.

2. How I broke it down

The first thing I did was stop treating this as one problem. It's really three, stacked, and each one only matters depending on what the one before it said. To answer the brief for any clip, I need to know three things in order:

- Gender first — because a female voice gets bounced straight away.
- Then age — because that's what decides senior-or-not, and whether emotion even runs.
- Then emotion — but only if the voice is male and 60 or over.

So no single model does this. It has to be a pipeline of three models with a bit of routing logic between them. Working that out early was honestly the most useful thing I did, because it turned a confusing one-liner of a brief into three jobs I could actually tackle one at a time.

I also decided early not to predict an exact age in years from the classical model. Even serious research systems are off by several years on voice-age, so trying to nail an exact number with hand-built features was never going to work. Instead I predicted age groups, which the dataset already provides, and which is all the senior rule actually needs. The routing logic ended up looking like this:

```
if gender == 'female':
    say 'Upload male voice.'      # stop here
else:
    # male
    age = predict_age(voice)
    if age is senior (60+):
        emotion = predict_emotion(voice)
        show senior + emotion
    else:
        show age group
```

3. Phase 1 — the classical pipeline (app.py)

My first version, which lives in app.py, is built entirely from classical machine learning. I pull a set of acoustic features out of each clip by hand with librosa, then train Random Forest classifiers on them. I went this way on purpose: it's light, it trains in minutes on a normal CPU, I can see which features matter,

and it forced me to actually reason about what separates the classes — which is the skill the brief was testing. All three models in Phase 1 share this same approach.

3.1 Gender model

Gender was the easy one, because pitch does most of the work — men's voices sit much lower than women's. For each clip I extract 13 MFCCs (a compact summary of the voice's tone), the average pitch using librosa's pyin, and a few spectral measures. Then a Random Forest learns the rest.

```
def extract_features(file_path):
    y, sr = librosa.load(file_path, sr=16000)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
    f0, _, _ = librosa.pyin(y, fmin=..., fmax=...) # pitch
    # + spectral centroid, bandwidth, zero-crossing rate
    return feature_vector
```

This landed around 89–90% on held-out data, which is solid. The one wrinkle was that it did better on male voices than female ones, and that comes straight from the data — Common Voice has a lot more labelled male clips than female. I tried class weighting and a richer feature set with jitter and shimmer plus some audio augmentation to fix it, but the augmented version did worse on real voices, so I threw that out and kept the simple model. That was a deliberate call, not laziness — I measured it and the simpler one won.

3.2 Age model

Age was where things got hard, and where most of my effort went. I trained only on male clips, since that's the only path that reaches the age stage. I kept the four working-age decades as their own classes (twenties, thirties, forties, fifties) and collapsed everything 60-and-up into a single senior class, because the senior flag is all the brief actually needs from the older end, and it merged a bunch of tiny, data-starved buckets into one bigger one.

```
SENIOR_SOURCE = {'sixties', 'seventies', 'eighties', 'nineties'}
KEEP_BUCKETS = {'twenties', 'thirties', 'forties', 'fifties'}
# 60+ -> 'senior_60plus', keep the rest as-is, drop teens
```

The big problem here is imbalance — the twenties and thirties buckets dwarf everything else. My first attempt, on a plain random sample, scored about 53% and was hopeless on the rare classes. The fix that actually moved the needle was per-class sampling: take every example I can get from the rare buckets, and cap the common ones. All real data, no synthetic anything.

```
def build_balanced_sample(df, max_per_class=1500):
    parts = []
    for label, grp in df.groupby('age_label'):
        parts.append(grp.sample(min(len(grp), max_per_class)))
    return pd.concat(parts)
```

That pushed it to about 63%, confirmed with 5-fold cross-validation that barely moved between folds, so it's a stable number not a fluke. The senior class specifically hit around 80% precision — when it says senior, it's usually right, which is the property that matters for gating emotion. Its recall was lower, meaning it misses some real seniors and calls them younger. That's a real weakness, but a safe one — it under-flags rather than over-flags.

```

Accuracy: 0.6453333333333333
      precision    recall  f1-score   support

   fifties         0.71      0.77      0.74        300
  fourties         0.63      0.69      0.66        300
senior_60plus       0.71      0.81      0.76        300
   thirties         0.56      0.51      0.53        300
   twenties         0.57      0.45      0.50        300

 accuracy          0.65          1500
  macro avg         0.64          1500
  weighted avg         0.64          1500

[[232  20  13  15  20]
 [ 19 207  16  35  23]
 [ 12  10 243  17  18]
 [ 33  41  33 152  41]
 [ 31  48  36  51 134]]
Classes: ['fifties' 'fourties' 'senior_60plus' 'thirties' 'twenties']

CV accuracy: 0.633 ± 0.008

```

3.3 Emotion model

Common Voice has no emotion in it — people just read sentences — so I used a different dataset, CREMA-D, where actors perform emotions and the label is baked into the filename. I picked CREMA-D over easier options like TESS on purpose: TESS only has two speakers and gives a fake-looking 99%, while CREMA-D has 91 speakers of both genders and a wide age range, which is much closer to the senior-male voices this stage actually sees.

Emotion lives in how a voice moves, not just its average, so I used a richer feature set here — 40 MFCCs with both their mean and standard deviation, deltas, chroma and mel features, plus energy and pitch stats. 163 features in total. I also scoped the emotions down. A four-class version including neutral got stuck around 71% because neutral is muddy and bleeds into everything. Dropping neutral and keeping the three clear ones — angry, happy, sad — pushed it to about 77%. I'm being upfront that this is a three-class result, scoped on purpose.

I tried augmentation here too, the proper way (only on the training split, after the split, so nothing leaks). It still made things slightly worse, so I reverted it — same lesson as the gender model.

[emotion confusion matrix (angry / happy / sad) + classification report]

```
Accuracy: 0.7745740498034076
      precision    recall  f1-score   support

   angry         0.77       0.79       0.78        255
   happy         0.68       0.65       0.67        254
   sad           0.87       0.88       0.87        254

 accuracy
macro avg         0.77       0.77       0.77        763
weighted avg         0.77       0.77       0.77        763

[[202  50   3]
 [ 57 166  31]
 [   2  29 223]]
Saved emotion model + scaler
```

3.4 How well Phase 1 actually worked

Honestly? Gender was good, emotion was decent, and age was the weak link. On the dataset's own clips it behaved fine, but the moment I tested it on real voices — playing an old man's voice off YouTube through a phone speaker, recording myself — it fell apart. It called clearly elderly men female, and put my own 21-year-old voice in the 40s. That's not a coding bug. It's two real problems: phone-speaker audio loses the low frequencies that mark a voice as male, and the models only ever saw clean dataset clips so anything else is foreign to them. Which leads straight into the next section.

4. Why Phase 1 wasn't good enough

I want to be straight about the ceiling on the classical version, because it's mostly baked into the problem, not something I did wrong.

First, age-from-voice is just hard. Gender has a clean signal in pitch, which is why it hits 90%. Age doesn't — two people of very different ages can sound similar, and neighbouring decades overlap even to a human ear. Research using far heavier deep-learning methods still lands at roughly 7 to 11 years of average error. Against that, my 63% grouped accuracy is about what this kind of approach can do, not a sign something's broken.

Second, the data is thin where it hurts. The labelled part of Common Voice is mostly young adults, and elderly male voices — exactly what the senior flag depends on — are among the rarest. My sampling squeezes the most out of what exists, but it can't invent older voices. The free datasets with reliable age labels for older speakers basically don't exist; the good ones (TIMIT, NIST) are behind licences I couldn't get.

Third, real-world recordings wreck it. Everything was trained on clean dataset audio. A phone speaker, a laptop mic, a compressed YouTube clip — all sound different enough that the models misread them, especially at the gender gate. This is the standard problem with any model trained on one source, and it's the main reason the live demo was unreliable even though the test numbers were fine.

My mentor looked at this and told me to take it to a deep-learning model. That's Phase 2.

5. Phase 2 — the deep-learning version (app2.py)

5.1 What wav2vec2 is and why I moved to it

My classical models rely on features I designed by hand. A deep model does something different: it learns its own features straight from the raw audio waveform, because it's been pre-trained on a huge amount of speech first. wav2vec2 is one of these — a transformer that takes raw audio and produces its own internal representation, which then feeds small prediction heads for age, gender or emotion.

I used pretrained wav2vec2 models published by audEERING — one for age/gender, one for emotion. I want to be completely honest about this: I did not train these models. I integrated them. Loading and running a state-of-the-art transformer, getting its inputs and outputs right, and then critically comparing it against my own work is real engineering, and that's what I'm claiming — not that I built the transformer. My self-built classical pipeline is still my core deliverable; the deep model is an upgrade I bolted on and benchmarked.

These models are pulled from Hugging Face by name at runtime, so there's nothing to download into the project folder — the weights come down on first run:

```
AGE_MODEL = 'audeering/wav2vec2-large-robust-6-ft-age-gender'  
EMO_MODEL = 'audeering/wav2vec2-large-robust-12-ft-emotion-msp-dim'  
# loaded with transformers + torch, run on CPU or GPU
```

5.2 Plugging it in

I kept my own gender model as the gate, because it works well and it's mine, and because the deep age model's gender output looked unreliable on my test clips anyway. So the hybrid in app2.py is: my classical Random Forest decides male/female, then if it's male the wav2vec2 model estimates the age in years, and if that age clears the senior line the wav2vec2 emotion model runs.

One thing worth flagging: the deep age model tends to underestimate older voices (it called several real seniors mid-40s). To compensate, I dropped the senior threshold from 60 to 52 in the app — a deliberate calibration, not a hidden hack, to catch more genuine seniors at the cost of a few false positives.

The emotion model also doesn't output words. It outputs three numbers — arousal, dominance and valence — which is the dimensional model of emotion used in research. I wrote a small mapping that turns those three numbers into a plain label (angry / happy / sad / neutral) for the interface, and the app shows both the label and the raw scores.

5.3 Benchmarking it against my own model

This is the part I think is most worth showing. I ran the deep age model on real Common Voice clips and compared its predictions to the true ages. My first instinct was to score it the same way as my classical model — bucket accuracy — and it came out at 38%, which looked terrible. But that's the wrong ruler: the deep model predicts a continuous age, and hard decade boundaries punish it brutally (a true fifties person predicted as 48 counts as a complete miss even though it's two years off).

Measured properly, with mean absolute error — the metric the research actually uses — it came out at 10.6 years, right inside the 7–11 year range that published wav2vec2 results report. So the deep model is genuinely good at general age estimation and clearly stronger than my classical one in that sense.

But here's the honest twist: for my specific job — catching seniors — even this model struggled, because it systematically underestimates elderly voices. Two completely different model types, my Random Forest and a deep transformer, both hit the same wall on old voices. That tells me the bottleneck isn't my modelling — it's a fundamental limit of reading age from voice for older speakers, and that's a stronger conclusion than any single accuracy figure.

Age model	How it was measured	Result
Phase 1 — my Random Forest	Bucket accuracy (5 classes)	~63% (stable across 5-fold CV)
Phase 2 — wav2vec2 (audEERING)	Mean absolute error (years)	10.6 years (in line with research)
Both	Catching real seniors (60+)	Weak — elderly voices underestimated

6. The two apps, side by side

I ended up with two working applications, and I'm keeping both on purpose — they show different things.

	app.py (Phase 1)	app2.py (Phase 2)
Gender	My Random Forest	My Random Forest (same model)
Age	My Random Forest (buckets)	wav2vec2 (age in years)
Emotion	My Random Forest (3-class)	wav2vec2 (dimensional)
Needs	Just scikit-learn + librosa	Also torch + transformers
Speed	Instant	Slower (deep models, esp. on CPU)
What it proves	I can build a pipeline from scratch	I can integrate and benchmark SOTA

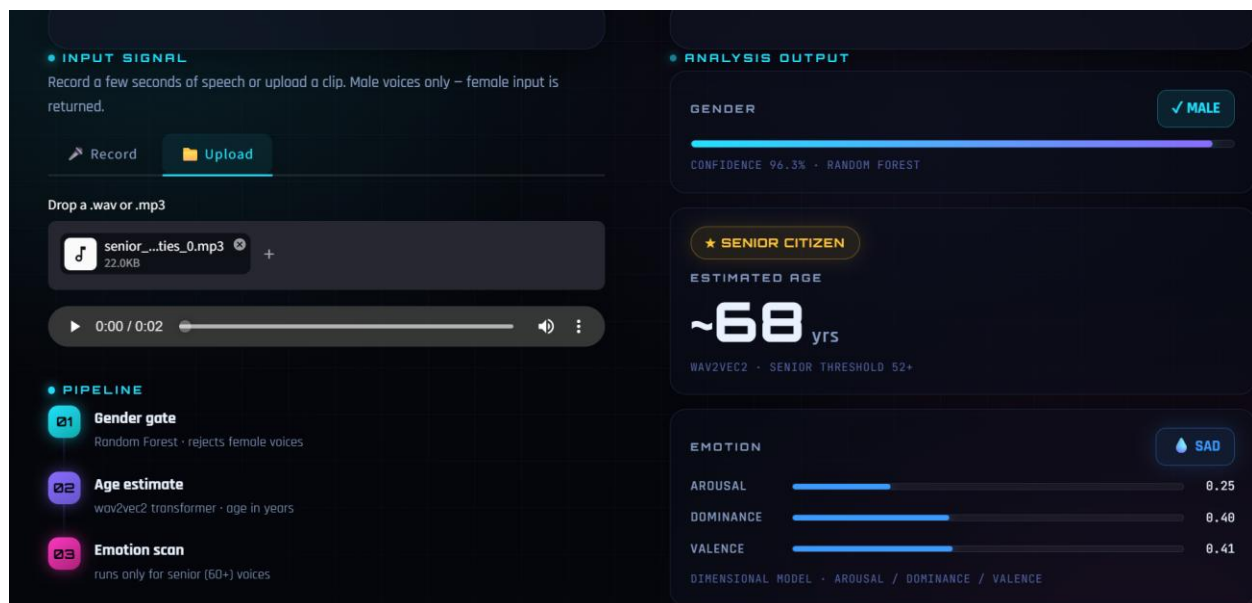
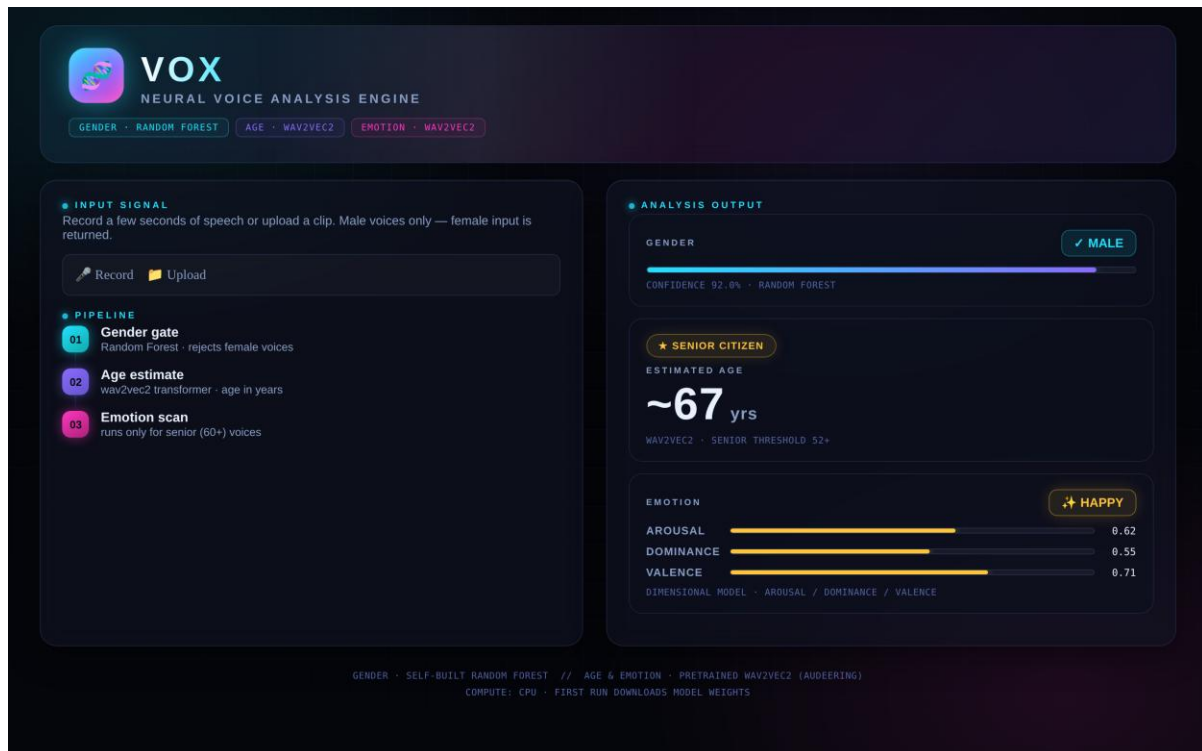
app.py is fully my own work and runs anywhere with no heavy dependencies. app2.py is the hybrid that swaps in the deep models for age and emotion. Together they make the point better than either alone: I built the thing from scratch, and I also know when and how to reach for a stronger pre-trained model.

7. The interface

The GUI is built in Streamlit, which turns a Python script into a web app without much fuss. It has two input modes — record straight from the mic, or upload a file — and runs the clip through the pipeline, showing each stage's result. Female voices get the rejection message; male voices show an age, and seniors additionally show emotion.

For app2.py I redesigned the interface into a full-width "neural voice analysis" console: a two-column layout with the input and a numbered pipeline rail on the left, and a live results panel on the right that

fills in stage by stage. The result cards show the gender with a confidence bar, the age as a large readout with a senior badge, and the emotion as a colour-coded label plus bars for the three dimensional scores.



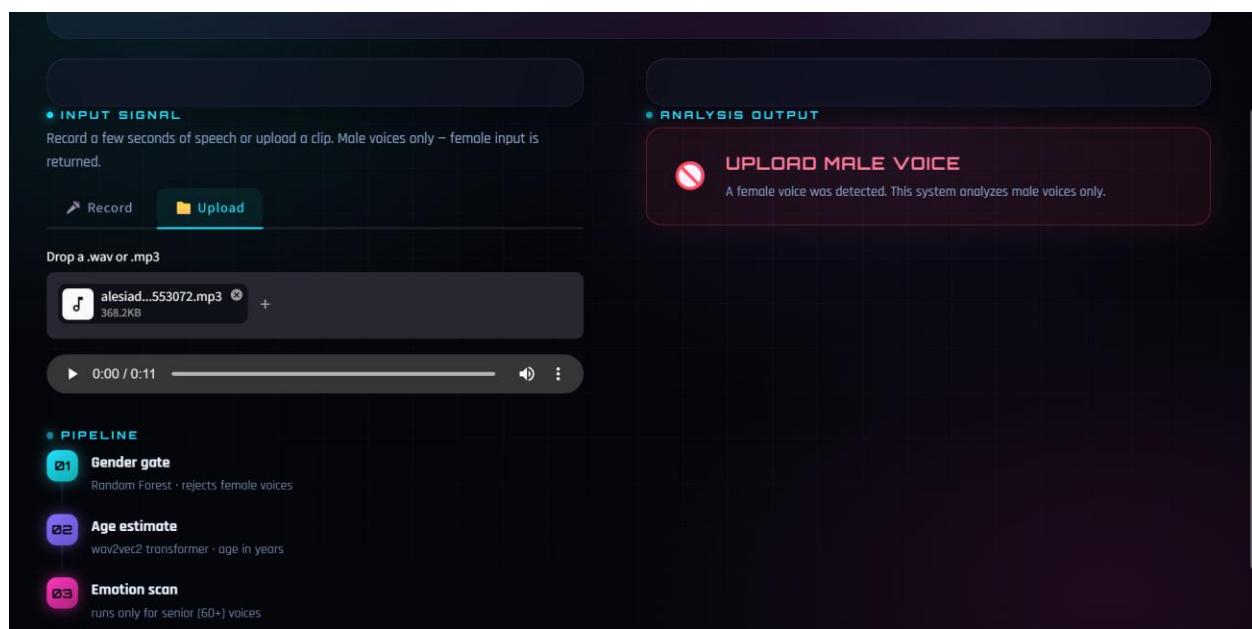
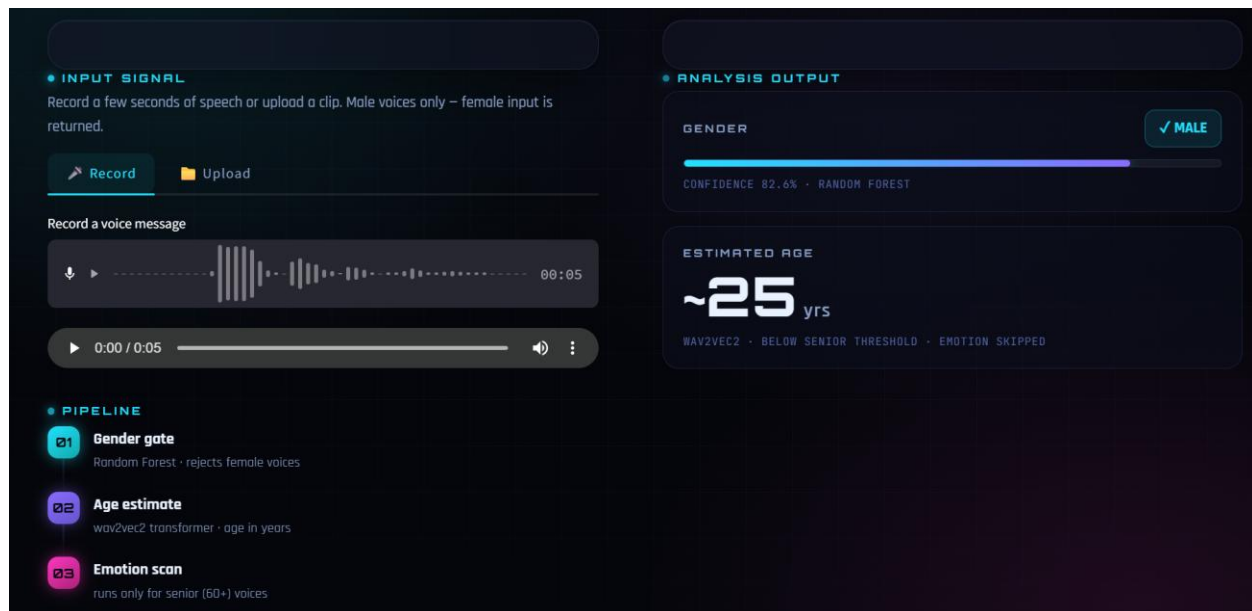


Figure — the Phase 2 interface (app2.py), shown with sample senior + emotion output.

One implementation detail that matters: the gender/age path and the emotion path use different feature pipelines (the emotion model needs a much richer input), so I keep them strictly separate. Feeding the wrong feature vector to a model doesn't error — it just silently produces nonsense — so this separation is deliberate.

8. Things that went wrong along the way

Class imbalance

The biggest one. Common Voice is mostly young adults and mostly male, so the older buckets are tiny. Per-class sampling across every split was how I got the most out of what existed.

Losing long extraction runs

Pulling features from thousands of clips is slow, and the pitch detector is the slowest part. I lost an early multi-hour run when the session disconnected before it saved anything. After that I cached features to CSV and checkpointed the loop every few hundred rows, so an interruption costs almost nothing.

Augmentation that backfired

I tried pitch-shift augmentation twice, for gender and emotion, expecting it to help the weak classes. Both times it made real-world performance worse, so I reverted both. The lesson I took: more engineering isn't automatically better, and every change has to be measured, not assumed.

Getting deep learning to run locally

Installing torch and transformers on my Windows/Anaconda setup was genuinely painful — build errors, a broken half-installed torch, version clashes. I fixed it by making a clean conda virtual environment and installing pinned, compatible versions into it, away from the tangled base environment. That isolation is what finally made app2.py run.

9. Results at a glance

Part	What it does	Result
Gender (mine)	Male/female gate	~89–90% test accuracy
Age (mine, Phase 1)	Age buckets + senior flag	~63%, stable on 5-fold CV
Emotion (mine, Phase 1)	Angry / happy / sad	~77% on CREMA-D (3 classes)
Age (wav2vec2, Phase 2)	Age in years	10.6 yr MAE (research-grade)
Whole system	Gate → age → senior → emotion	Correct routing, shown in both GUIs

10. Where I landed, and what's next

This started as one confusing sentence and turned into a small system. The real insight was seeing it as three gated stages instead of one model, and a lot of the work was in wiring them together and then being honest about where each one breaks.

Phase 1 is my own pipeline, built from scratch — gender, age and emotion on hand-made features and Random Forests, with a working GUI. It does what the brief asks. Phase 2 is the upgrade my mentor pushed me toward: I swapped in pretrained wav2vec2 models for age and emotion, got a research-grade 10.6-year age error, and built a benchmark comparing it honestly against my own model. Along the way I learned the same thing twice — that augmentation can quietly hurt — and learned that the elderly-voice problem is a real ceiling that even a deep transformer doesn't escape.

If I kept going, the next steps are clear: get a bigger, age-balanced dataset of older speakers (the real bottleneck), maybe fine-tune the wav2vec2 model myself rather than using it as-is, and add some noise-robust preprocessing so the live demo holds up better on phone and web audio. But as it stands I've got two working apps, three self-built models, a deep-learning benchmark, and a clear, honest account of what works and what doesn't — which is what I set out to deliver.